

LECTURES – 27..30

STRING MATCHING

Informally, we wish to find all the occurrences of pattern string P in text T .

Text $T[1..n]$ is a character array of length n .

Pattern $P[1..m]$ is a character array of length $m \leq n$.

Every character in T and P is drawn from **alphabet** Σ
Both T and P are known as **strings** over the alphabet.

Pattern P is said to occur with **shift** s in text T if:

$$0 \leq s \leq n-m,$$

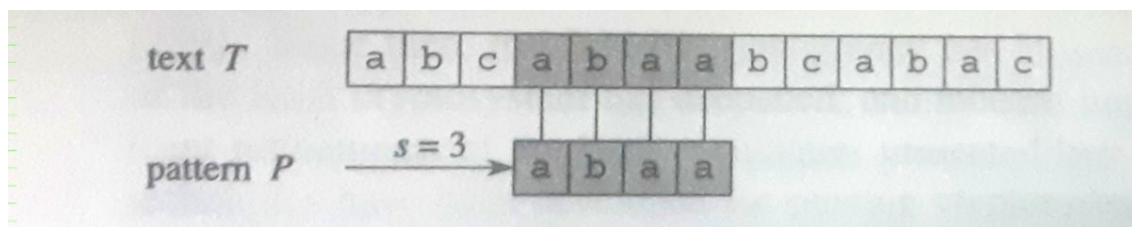
and

$$T[s+j] = P[j] \quad \dots \text{ for } 1 \leq j \leq m.$$

If P occurs with shift s in T , we say s is a **valid shift** of P in T .

String matching problem: Find all the valid shifts of pattern P in text T .

Example:



Here $n = 13$, $m = 4$ and the only valid shift is $s = 3$.

String matching algorithms are compared on the basis of **running time**, but a so-called **pre-processing time** is also usually considered. See below.

Algorithm	Preprocessing time	Matching time
Naive	0	$O((n - m + 1)m)$
Rabin-Karp	$\Theta(m)$	$O((n - m + 1)m)$
Finite automaton	$O(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$

Notation:

Σ^* is the set of all strings formed over alphabet Σ , including ε , the empty string. Note that Σ^* is a countably infinite set.

Concatenation of two strings x and y , denoted xy , has the usual meaning.

$|x|$ denotes the length of string x .

$w \sqsubset x$ indicates that string w is a **prefix** of string x .

$w \sqsupset x$ indicates that string w is a **suffix** of string x .

The empty string ε is a prefix and a suffix of every string.

Overlapping suffix lemma:

Let x , y and z be strings such that both x and y are suffix strings of z ; that is, $x \sqsupset z$ and $y \sqsupset z$.

If $|x| \leq |y|$, then $x \sqsupset y$.

If $|x| \geq |y|$, then $y \sqsupset x$.

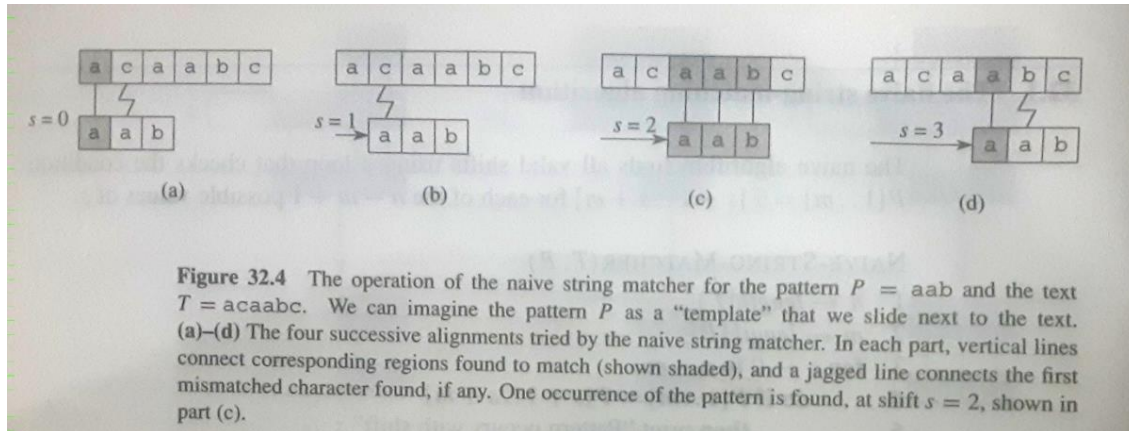
If $|x| = |y|$, then $x = y$.

[This lemma can be understood easily with the help of a simple diagram.]

Notation: The prefix of length k of string P will be denoted as P_k .

NAIVE-STRING-MATCHER (T, P)

```
n = length[T];  
m = length[P];  
for s = 0 to n-m  
    if P[1..m] = T[s+1..s+m]    // Note array  
                                // comparison  
        print "Pattern occurs with shift" s
```



In this case, **running time = matching time** : $O((n-m+1)m)$

[No pre-processing is required, and therefore running time equals matching time.]

Food for thought: 🍔 🍔 🍔

Suppose $P = "aaab"$ and $s = 0$ is a valid shift. Then $s = 1, 2$ & 3 **cannot be** valid. In other words, **information gained** from $s = 0$ can be used later.

Think of this as a 'general principle'.

RABIN-KARP ALGORITHM

Main idea: Any m consecutive characters, in any string, define an m -digit number in base $|\Sigma|$ arithmetic. **Example:** If characters are 8 bits each, then $|\Sigma| = 256$. So the string 'ABC' defines as a 3 digit number, with base 256. This string corresponds to integer:

`ascii['A']*256^2 + ascii['B']*256 + ascii['C']`

In this way, the pattern string $P[1..m]$ represents an m digit number with base $|\Sigma|$. This insight helps achieve efficiency in string matching.

This algorithm also makes use of **modulo q arithmetic** (for a fairly large prime number q). Example, with small $q = 19$:

$$100 \text{ modulo } 19 = \text{remainder}(100/19) = \text{mod}(100,19) = 5.$$

A simple way to understand **modulo q arithmetic** is on a circle with q sub-divisions of the circumference. Zero is shown at the top, and numbers increase clockwise. Therefore **$(q-1)$** appears on the circle to the left of zero. Arithmetic operations can be performed on the set of integers $\{ 0, 1, \dots, (q-1) \}$.

Another quick reference: Horner's rule for evaluating a polynomial:

Let $p(x)$ be a polynomial of degree k . Then $p(x)$ can be evaluated efficiently as:

$$p(x) = ((a_k * x + a_{k-1}) * x + a_{k-2}) * x + a_{k-3} \dots) * x + a_0$$

This process requires k multiplications, as against $k*(k+1)/2$. These k multiplications can be carried out iteratively, as in the code below.

Finally, we must remember that infinitely many unequal integers will come out equal in modulo q arithmetic. Example:

$$\text{mod}(\textcolor{red}{100},19) = \text{mod}(\textcolor{red}{119},19) = \text{mod}(\textcolor{red}{138},19) = \dots = 5$$

This means that:

1. Two different strings of length m can map to the same numeric value, modulo q .
2. But if the numeric values of two strings are unequal, the two strings cannot be equal.

Inequality of numeric values tells you that the strings are unequal. But equality of numeric values does not guarantee that the strings are equal. However, if q is 'sufficiently large', the probability that unequal text strings generate equal numeric value is very, very small.

This observation leads to the analysis of running time. Under reasonable assumptions, the expected running time reduces to $\Theta(n)$.

RABIN-KARP-MATCHER(T,P,d,q)

// The following are integers

n = length[T];

m = length[P];

h = mod($d^{(m-1)}$, q);

p = 0;

t₀ = 0;

// Pre-processing -- Horner's rule applied twice

for i = 1 to m

p = mod(d*p+P[i], q); // 'integerized' P

t₀ = mod(d*t₀+T[i], q); // ... and T[1..m]

// Matching

for s = 0 to n-m

// Note loop variable in subscript!!

if p = t_s // If numeric values are equal ..

if P[1..m] = T[s+1..s+m] // .. confirm equality

print "Pattern occurs with shift s"

// Now we update t_s

if s < n-m

t_{s+1} = mod(d*(t_s - T[s+1]*h) + T[s+m+1], q);

Recall that the first character of text being matched is T[s+1]. We are moving the m character 'window' in text forward by one character.

FINITE AUTOMATON

A theoretical model of computation, which can in practice be realized in a digital circuit or in software. The properties and techniques which we establish for the model carry over automatically to all its realizations.

Definition: A finite automaton M is a 5-tuple $M = (Q, q_0, A, \Sigma, \delta)$ where

Q is a finite set of states,

$q_0 \in Q$ is the start state,

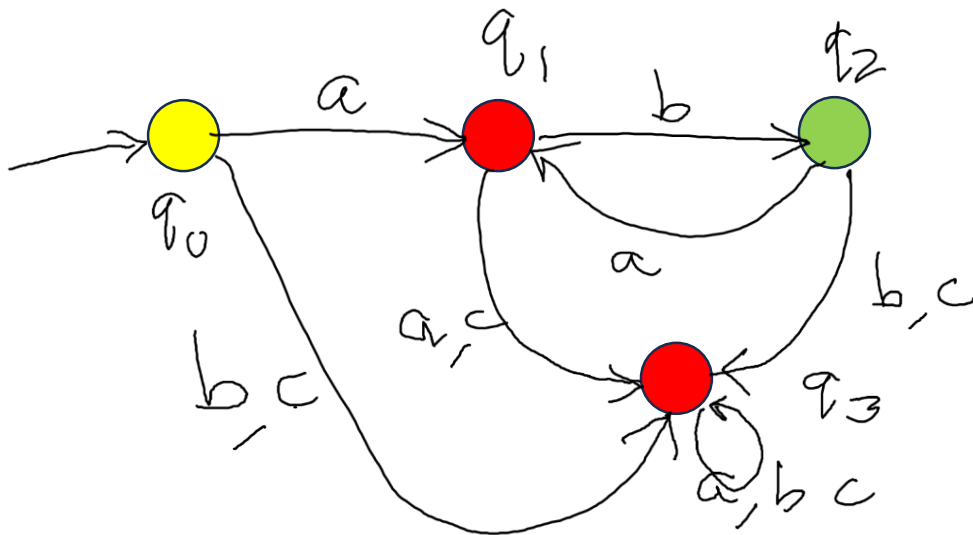
$A \subset Q$ is a set of accepting states,

Σ is the input alphabet, and

δ is the transition function of M mapping $Q \times \Sigma$ into Q .

Given present state and input symbol, δ specifies the next state.

Example: Finite automaton which recognizes $(ab)^+$, with $\Sigma = \{a, b, c\}$



Note that this is a **graphical representation** of this particular **discrete mathematical structure**. Here q_2 is the only one accepting state; q_1 and q_3 can be seen as “rejecting states”. [Why? Think about when the input ends.]

We can easily specify the function δ in tabular form.

Present state	Input symbol	Next state
q_0	a	q_1
q_0	b	q_3
...	...	...
...	...	...

Number of rows = $|Q| \times |\Sigma|$

Using the FA, the computing problem can be posed in TWO ways:

1. Does the given input string match the pattern defined by the FA?
2. Count the number of times the given pattern string, defined by the FA, occur in the (indefinitely long) string which we call **text**.

In the first form, the FA can be used for 'syntax checking' a given string. In the second form, the FA helps find all occurrences of P in T. In this chapter, we are looking at the second form of the problem, even though the example FA drawn above corresponds to the first form of the problem.

GK: The number of states $|Q|$ can be minimized, but that is not our concern at present – as long as M performs as it is expected to do.

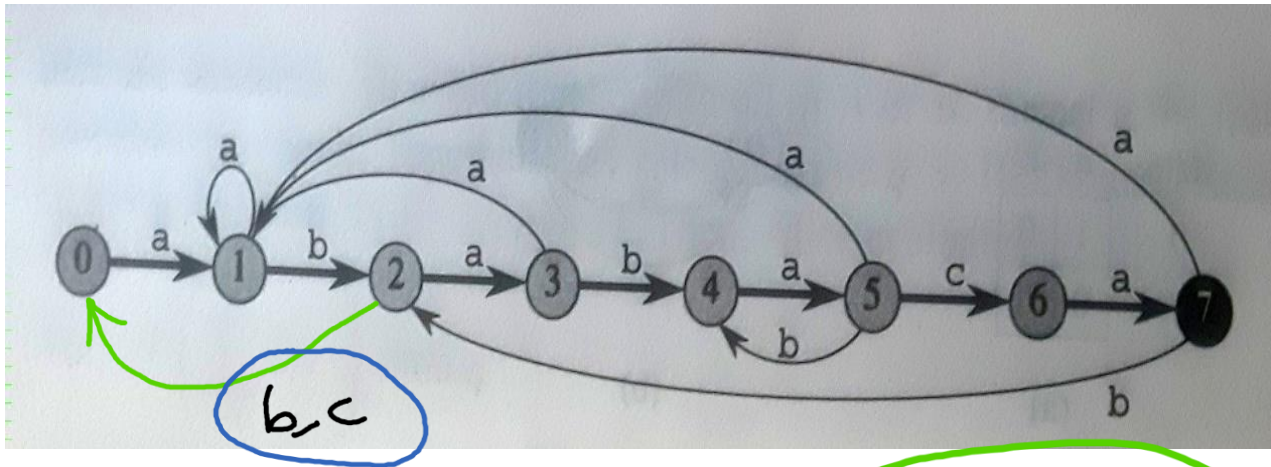
The automaton with greater 'computing power' than the FA is the **push-down automaton (PDA)**, which has a stack. The type of automaton which is 'as powerful as any computer program' is the **Turing machine**. These topics are introduced in a course such as **Formal Languages and Automata**.

Think about this: Given an FA in tabular form, how will you 'execute' or 'simulate' it in software?

Example [CLRS]:

FA to find all occurrences of "ababaca" in text.

Key question: How do we build this FA algorithmically?



q_0 : matched nothing yet = P_0
 q_1 : matched "a" = P_1
 q_2 : matched "ab" = P_2
 q_3 : matched "aba" = P_3
 q_4 : matched "abab" = P_4
 q_5 : matched "ababa" = P_5
 q_6 : matched "ababac" = P_6
 q_7 : matched full P = P_7

Edges not shown explicitly
all lead to state $q_0 \rightarrow$ i.e.
"reset". See example.

In short, state q_k corresponds to having matched P_k .

Say you have matched P_k . Then, **by definition**, the last k characters of T read so far match P_k . Thus P_k is a suffix of the string read so far, that is: $P_k \sqsupset T_{(\text{so far})}$.

The way in which states have been numbered $[0..m]$, we can also say that the state q_k corresponds to having matched prefix P_k of P .

Consider the following **suffix function**, defined w.r.t. the pattern P :

$$\sigma(x) = \max(k : P_k \sqsupset x) \quad \text{where } x \text{ is any string, } x \in \Sigma^*.$$

$\rightarrow \sigma(x)$ is the length of the longest prefix of P which is a suffix of string x . In other words, after scanning string x (as "front part" of text T), the FA would be in state $q_{\sigma(x)}$, having matched the prefix P_k .

Key question: How to compute the **transition** table? **Answer:** $\delta(q, a) = \sigma(P_q a)$. A simple, "brute force" approach is taken in the algorithm below:


```

// Note global variables

string P;
m = length[P];
int array TRANSITION-TABLE[ m+1, | $\Sigma$ | ];

COMPUTE-TRANSITION-FUNCTION()
  for  $q = 0$  to  $m$ 
    for each  $a$  in  $\Sigma$ 
       $k = \min( m+1, q+2 );$ 
      repeat
         $k = k-1;$           // To find  $\max(k:\dots)$  as above
      until  $P_k \sqsupseteq P_q a;$  // <-- naive string matching
      // Found  $\sigma$ , which is  $\max(k:\dots)$ 
      TRANSITION-TABLE[ $q, a$ ] =  $k;$ 

// Note: Here T is an argument, P is not

FINITE-AUTOMATON-MATCHER(T)
   $n = \text{length}[T];$ 
   $q = 0;$ 
  for  $i = 1$  to  $n$ 
     $q = \text{TRANSITION-TABLE}[ q, T[i] ];$  // Transition
    if  $q = m$ 
      print "Pattern occurs with shift",  $i-m;$ 

```